Cahier de charge Équipe DevLeaders – MIABE Hackathon 2026

CAHIER DES CHARGES – PROJET CHAIN CACAO

Traçabilité café/cacao par blockchain – Togo

Version : 2.1 – Architecture Hyperledger Fabric
Hackathon : MIABE 2026 – DAROLLO TECHNOLOGIES CORPORATION
Stack technique : Hyperledger Fabric · Go · Next.js · React Native

SOMMAIRE

1. Présentation générale
2. Contexte et enjeux
3. Objectifs du projet
4. Utilisateurs finaux
5. Architecture technique
6. Fonctionnalités détaillées
7. Modèle de données
8. Smart contracts
9. API Backend
10. Frontend Web (Next.js)
11. Application Mobile (React Native)
12. Exigences non fonctionnelles
13. Contraintes spécifiques
14. Livrables par phase
15. Critères d'acceptation
16. Indicateurs de performance
17. Planning de déploiement

1. PRÉSENTATION GÉNÉRALE

Élément	Description
Nom du projet	ChainCacao
Pays cible	Togo
Secteur	Agriculture – Filière café/cacao
Problème principal	Absence de traçabilité entraînant pertes financières, fraudes et non-conformité au règlement européen EUDR
Solution	Plateforme blockchain de traçabilité de la ferme jusqu'à l'exportation

Élément	Description
Valeur ajoutée	Preuve d'origine immuable, accès aux marchés premium, réduction des fraudes sur les pesées

2. CONTEXTE ET ENJEUX

2.1 Constats chiffrés

Indicateur	Valeur
Familles dépendant du café/cacao	Plus de 40 000
Part du revenu agriculteur dans la valeur finale	15 à 25 %
Exportations certifiées bio ou Fairtrade	Moins de 5 %
Pertes annuelles dues aux fraudes sur les pesées	30 à 40 millions USD
Intermédiaires entre la ferme et l'exportateur	4 à 6 non tracés

2.2 Contrainte réglementaire

Le **règlement européen EUDR (2025)** impose une **traçabilité géographique prouvée** pour tout cacao importé dans l'Union Européenne. Cette preuve doit démontrer que le produit n'est pas issu de zones déforestées après 2020.

Conséquence : Sans solution de traçabilité fiable, le Togo risque l'exclusion de ses marchés d'exportation européens.

2.3 Problème central

Les agriculteurs togolais produisent du café et du cacao de qualité, mais ne disposent d'aucun moyen pour prouver :

- L'origine géographique précise de leur récolte
- Le parcours complet du produit
- Les conditions de production

Cette opacité les prive :

- Des certifications premium (bio, Fairtrade, Rainforest Alliance)
- D'une juste rémunération
- De l'accès aux marchés européens à partir de 2025

3. OBJECTIFS DU PROJET

3.1 Objectif général

Développer une solution blockchain permettant d'enregistrer, suivre et vérifier chaque lot de café ou cacao, depuis la ferme jusqu'à l'exportateur, pour satisfaire les exigences EUDR et permettre l'accès aux certifications premium.

3.2 Objectifs spécifiques

Code	Objectif	Description
OS1	Enregistrement sécurisé	Permettre à l'agriculteur d'enregistrer un lot avec ses caractéristiques (origine GPS, poids, espèce, date)
OS2	Suivi des transferts	Assurer le suivi du lot à chaque étape : ferme → coopérative → transformateur → exportateur
OS3	Vérification instantanée	Offrir à tout acheteur la possibilité de vérifier l'historique complet via un QR code
OS4	Conformité EUDR	Générer automatiquement une preuve de traçabilité acceptable par l'Union Européenne
OS5	Accessibilité terrain	Fonctionner en mode hors ligne pour les zones rurales sans connectivité
OS6	Compatibilité mobile	Être utilisable sur smartphones Android d'entrée de gamme

4. UTILISATEURS FINAUX

Acteur	Rôle principal	Interface utilisée
Agriculteur	Enregistre un lot récolté	Mobile (React Native)
Coopérative	Regroupe et transfère les lots	Web (Next.js) + Mobile
Transformateur	Réceptionne et transforme les lots	Web (Next.js)
Exportateur	Prépare l'export avec preuve EUDR	Web (Next.js)
Vérificateur (acheteur, douane)	Consulte l'historique d'un lot via QR code	Web (Next.js - sans authentification)
Ministère de l'Agriculture	Supervise et audite la filière	Web (Next.js - backoffice)
Organismes de certification	Vérifient la conformité pour les labels	Web (Next.js)

5. ARCHITECTURE TECHNIQUE

5.1 Stack technologique

Couche	Technologie	Justification
Blockchain	Hyperledger Fabric 2.5+	Blockchain permissionnée, adaptée aux consortiums d'acteurs ne se faisant pas confiance
Smart contracts	Go (chaincode)	Langage natif de Fabric, performances optimales, support officiel
Backend API	Go (fabric-sdk-go)	SDK officiel Fabric, intégration native, un seul langage pour toute la couche serveur
Base de données	PostgreSQL (données off-chain) + CouchDB (état monde Fabric)	Stockage des métadonnées utilisateurs et cache
Frontend Web	**Next.js 14+ (App Router)**	Framework React avec SSR, SEO optimisé pour page de vérification publique, routing intégré
Application mobile	React Native + Expo	Réutilise la logique web, mode hors ligne natif
Cache mobile	SQLite	Stockage local pour fonctionnement hors ligne
Conteneurisation	Docker + Docker Compose	Déploiement standard de Fabric
Orchestration	Kubernetes (optionnel pour production)	Scalabilité et haute disponibilité

Pourquoi Next.js plutôt que React seul ?

Fonctionnalité	React seul	Next.js	Bénéfice pour ChainCacao
Rendu côté serveur (SSR)	Configuration manuelle	Natif	SEO optimal pour page de vérification publique
Rendu statique (SSG)	Non	Natif	Page d'accueil ultra-rapide
API routes	Nécessite serveur séparé	Intégré	Endpoints de vérification publique sans backend dédié
Optimisation images	Manuel	`next/image`	Photos de lots optimisées
Internationalisation	Librairie externe	`next-intl` intégré	Support français/anglais
Déploiement	Build + serveur séparé	Vercel / auto-hébergé	Simplicité

5.2 Organisation du réseau Fabric

Principe : Le réseau repose sur une blockchain permissionnée où chaque type d'acteur constitue une organisation distincte.

Organisations définies :

Organisation	Acteurs concernés	Rôle dans le réseau
Org1 – Agriculteurs	Producteurs individuels ou groupés	Création des lots de récolte
Org2 – Coopératives	Agents coopératifs	Regroupement et premier transfert
Org3 – Transformateurs	Usines de transformation	Réception et transformation
Org4 – Exportateurs	Sociétés d'exportation	Préparation des lots pour l'export
Orderer	Autorité indépendante	Consensus et ordonnancement des transactions

Configuration :

- Une **channel unique** nommée "chaincacao-channel" où toutes les organisations peuvent interagir
- Chaque organisation possède son propre **Certificate Authority (CA)** pour gérer ses identités
- Consensus **RAFT** pour la validation des blocs

Droits d'accès par acteur :

Action	Agriculteur	Coopérative	Transformateur	Exportateur	Ministère
Créer un lot	✅	❌	❌	❌	❌
Transférer un lot	❌ (après vente)	✅	✅	✅	❌
Consulter un lot	Ses propres lots	Lots entrants/sortants	Lots reçus	Lots à exporter	Tous les lots
Générer preuve EUDR	❌	❌	❌	✅	❌
Auditer	❌	❌	❌	❌	✅

5.3 Schéma d'architecture applicative

Architecture globale avec Next.js :

```
┌──────────────
```

```
│  │  ├── /verify/[id]     → Page vérification (SSR/ISR)          ||
│  │  ├── /dashboard       → Zone authentifiée (SSR)              ||
│  │  └── /api/*           → API Routes (proxy vers Go backend)   ||
│  └──────────────────────────────────────────────────────────────│
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
                        API Gateway Go (Port 8080)
```

6. FONCTIONNALITÉS DÉTAILLÉES

6.1 Module Agriculteur (Mobile React Native)

F-AGR-01 : Enregistrement d'un lot

Élément	Description
Acteurs	Agriculteur
Déclencheur	Fin de récolte
Précondition	Agriculteur identifié dans le système (ou mode invité avec enregistrement simplifié)
Postcondition	Lot créé avec identifiant unique et QR code associé

Fonctionnement détaillé :

L'agriculteur ouvre l'application mobile sur son smartphone. L'écran d'accueil affiche un bouton "Nouveau lot". Après avoir cliqué, un formulaire simplifié s'ouvre avec les champs suivants :

- **Type de produit** : choix entre Café ou Cacao (boutons radio)
- **Variété** : sélection dans une liste déroulante (Forastero, Trinitario, Robusta, etc.)
- **Poids** : saisie numérique en kilogrammes
- **Parcelle** : sélection dans une liste pré-enregistrée des parcelles de l'agriculteur

L'application capte automatiquement :

- Les coordonnées GPS via le capteur du téléphone
- La date et l'heure actuelles
- Une photo facultative du lot

Après validation, le système génère un identifiant unique au format standardisé. Un QR code correspondant est affiché à l'écran, que l'agriculteur peut imprimer (via impression Bluetooth ou sauvegarde PDF) pour l'attacher à son sac.

Mode hors ligne spécifique :

Situation	Comportement
Pas de connexion internet	Le lot est sauvegardé localement dans la base SQLite avec un statut "en attente de synchronisation"
Connexion rétablie	Synchronisation automatique en arrière-plan, sans action de l'utilisateur
Conflit de données	Le système conserve la version avec l'horodatage le plus récent

F-AGR-02 : Consultation des lots

L'agriculteur peut visualiser la liste de tous ses lots enregistrés. L'écran affiche pour chaque lot :

- L'identifiant unique
- La date de création
- Le poids
- Le statut (local uniquement, synchronisé, ou transféré)
- La couleur du statut (rouge, orange, vert)

Un filtre permet de trier par date, poids ou statut.

F-AGR-03 : Impression des QR codes

L'agriculteur peut, pour chaque lot, générer un QR code au format imprimable. Le QR code contient l'URL publique de vérification (ex: `https://chaincacao.tg/verify/TC-2026...`) ou directement l'identifiant du lot.

Format d'impression recommandé : Étiquette autocollante résistante à l'humidité, à fixer sur chaque sac.

6.2 Module Coopérative (Web Next.js)

F-COOP-01 : Réception de lots

Élément	Description
Acteurs	Agent coopératif
Déclencheur	Arrivée de sacs de cacao à la coopérative
Précondition	Agent authentifié sur la plateforme

Fonctionnement détaillé :

L'agent coopératif se connecte à l'application web Next.js. Il accède à la page `/dashboard/reception` qui intègre un lecteur de QR code utilisant la caméra du poste de travail (ou via un scanner USB).

Il scanne le QR code présent sur le sac de l'agriculteur.

Le système affiche automatiquement toutes les informations pré-enregistrées du lot : identifiant, agriculteur, origine GPS, poids déclaré, date de récolte.

L'agent procède alors à un contrôle :

- Il pèse à nouveau le sac sur la balance de la coopérative
- Il compare le poids avec celui déclaré
- Il vérifie visuellement la qualité

Il valide ensuite la réception. Le système enregistre alors le transfert de propriété dans la blockchain : l'agriculteur n'est plus propriétaire, la coopérative devient le nouveau propriétaire. Un événement de transfert est enregistré avec l'horodatage, l'identité de l'agent et le nouveau poids.

F-COOP-02 : Transfert vers le transformateur

Élément	Description
Acteurs	Agent coopératif
Déclencheur	Envoi d'un lot à un transformateur

Fonctionnement détaillé :

L'agent sélectionne un ou plusieurs lots dans son inventaire (cases à cocher). Il clique sur "Transférer" et sélectionne le transformateur destinataire dans une liste pré-enregistrée.

Il peut saisir un nouveau poids si un contrôle qualité a été effectué. Il valide le transfert. La transaction est enregistrée dans la blockchain, mettant à jour la propriété du lot vers le transformateur. Le QR code du lot est automatiquement mis à jour pour refléter le nouveau propriétaire.

Règle de gestion : Un lot ne peut être transféré qu'à un acteur légalement enregistré dans le système. Le transfert est irréversible.

6.3 Module Transformateur (Web Next.js)

F-TRA-01 : Réception et transformation

Élément	Description
Acteurs	Agent transformateur
Fonction	Confirmer la réception et enregistrer la transformation

Fonctionnement :

Le transformateur scanne les QR codes des lots entrants. Il confirme la réception. Si le lot subit une transformation (broyage, torréfaction, etc.), il peut diviser le lot d'origine en plusieurs sous-lots transformés. Chaque sous-lot hérite de l'historique parent et génère son propre identifiant.

F-TRA-02 : Transfert vers l'exportateur

Même mécanisme que le transfert coopérative → transformateur. Le transformateur sélectionne les lots finis et les transfère à un exportateur.

6.4 Module Exportateur (Web Next.js)

F-EXP-01 : Export et génération de la preuve EUDR

Élément	Description
Acteurs	Exportateur
Déclencheur	Préparation d'un conteneur pour l'exportation vers l'UE

Fonctionnement détaillé :

L'exportateur sélectionne les lots qu'il souhaite exporter (un ou plusieurs). Il clique sur "Générer rapport EUDR". Le système :

1. Extrait pour chaque lot : les coordonnées GPS de la parcelle source, les dates de récolte, et la chaîne complète des propriétaires successifs
2. Vérifie que chaque lot répond aux critères EUDR (absence de zone déforestée)
3. Génère un document PDF signé numériquement contenant toutes ces preuves
4. Ajoute un QR code unique sur le rapport permettant à la douane européenne de vérifier l'authenticité
5. Enregistre une transaction finale "exporté" dans la blockchain

Contenu du rapport EUDR :

- Identification du lot (identifiant unique)
- Coordonnées GPS de la parcelle d'origine avec précision
- Date de récolte
- Chaîne complète des propriétaires successifs avec horodatages
- Hash de la transaction blockchain comme preuve d'immutabilité
- Signature numérique de l'exportateur
- QR code de vérification (URL `https://chaincacao.tg/verify/...`)

Format de sortie :

- PDF signé (format standard européen)

- JSON structuré (pour intégration API douanière)

6.5 Module Vérificateur Public (Next.js - Page dédiée)

F-VER-01 : Vérification par QR code

Élément	Description
Accès	Aucune authentification requise, public
Déclencheur	Un acheteur ou douanier scanne un QR code
Rendu Next.js	SSR ou ISR (revalidation automatique)

Fonctionnement détaillé :

L'utilisateur scanne le QR code présent sur le sac ou sur le rapport EUDR. Le QR code contient l'URL complète : `https://chaincacao.tg/verify/TC-20260315-00123`

La page `/verify/[id]` de Next.js s'ouvre. Le rendu peut être :

- **SSR (Server-Side Rendering)** : À chaque chargement, l'appel API est fait côté serveur
- **ISR (Incremental Static Regeneration)** : Page statique régénérée toutes les X minutes

La page affiche automatiquement les informations suivantes, sans aucune exigence d'authentification :

Section 1 – Origine :

- Nom de l'agriculteur
- Coordonnées GPS (latitude/longitude)
- Village et région
- Parcelle
- Photo du lot (si disponible)

Section 2 – Parcours chronologique :
Une ligne du temps interactive montre chaque transfert avec :

- La date et l'heure
- L'acteur émetteur (type et nom)
- L'acteur récepteur
- Le poids à chaque étape

Section 3 – Conformité réglementaire :

- Statut de conformité EUDR (vert si conforme, rouge sinon)
- Horodatage de la dernière vérification
- Hash de la transaction blockchain (consultable sur un explorateur)

Section 4 – Export de l'historique :
L'utilisateur peut télécharger l'historique complet au format PDF ou CSV.

Avantage SEO Next.js : Chaque page de lot est indexable par les moteurs de recherche, permettant aux acheteurs internationaux de trouver la traçabilité des lots togolais.

6.6 Module Backoffice (Ministère - Next.js)

F-BO-01 : Tableau de bord de supervision

Fonction	Description
Volume tracé	Cartographie interactive du Togo montrant par région les tonnes de cacao enregistrées
Alertes	Détection automatique des lots sans transfert depuis plus de 30 jours
Conformité	Pourcentage des lots conformes EUDR par région
Fraudes détectées	Anomalies de pesée détectées par comparaison des poids entre étapes

Accès : Réservé au Ministère de l'Agriculture avec authentification forte.

F-BO-02 : Audit complet

L'administrateur peut rechercher n'importe quel lot par identifiant, agriculteur, ou période. Le système affiche l'historique complet immuable et permet d'exporter les données pour analyse.

7. MODÈLE DE DONNÉES

7.1 Données stockées SUR la blockchain (Ledger Fabric)

Les données suivantes sont écrites dans le ledger car elles doivent être immuables et vérifiables par tous :

Champ	Type	Description
Identifiant du lot	Chaîne	Format unique TC-AAAA-MM-JJ-NNNNN
Identifiant agriculteur	Chaîne	Référence unique du producteur
Nom agriculteur	Chaîne	Nom complet
Latitude GPS	Décimal	Coordonnée géographique (précision 8 décimales)
Longitude GPS	Décimal	Coordonnée géographique
Région	Chaîne	Plateaux, Maritime, Savanes, Centrale, Kara
Village	Chaîne	Nom du village
Type de produit	Chaîne	"cacao" ou "cafe"

Champ	Type	Description
Variété	Chaîne	Forastero, Trinitario, Robusta, etc.
Poids initial	Décimal	Kilogrammes
Certification bio	Booléen	Oui/non
Date de récolte	Date	Format AAAA-MM-JJ
Date de création	Horodatage	Premier enregistrement
Propriétaire actuel	Chaîne	Organisation détenant le lot
Statut	Chaîne	"créé", "transféré", "exporté"
Liste des transferts	Tableau	Liste chronologique des changements de propriété
Conformité EUDR	Booléen	Validé par le système
Hash de bloc	Chaîne	Preuve d'ancrage blockchain

Chaque transfert contient :

- Émetteur (identifiant et type d'organisation)
- Destinataire (identifiant et type d'organisation)
- Date et heure de la transaction
- Poids à l'étape
- Identifiant de la transaction blockchain

7.2 Données stockées HORS blockchain (PostgreSQL/CouchDB)

Table	Contenu	Justification
Utilisateurs	Email, mot de passe haché, rôle, organisation	Données confidentielles, volumes importants
Agriculteurs	Nom, village, pièce d'identité, numéro de téléphone	Pas besoin d'immutabilité
Coopératives	Nom, numéro d'enregistrement, adresse	Modification possible
Parcelles	Surface, culture principale, année plantation	Peut évoluer dans le temps
Appareils mobiles	Identifiant device, token de notification	Lié à l'utilisateur, pas à la traçabilité
File d'attente synchro	Lots créés hors ligne, statut, horodatage	Temporaire, supprimé après envoi
Photos des lots	URL des images stockées (Cloud Storage)	Trop volumineux pour blockchain

8. SMART CONTRACTS (CHAINCODE)

8.1 Rôle du chaincode

Le chaincode (smart contract) est le code exécuté sur la blockchain Fabric. Il est responsable :

- De valider que les transactions sont autorisées
- De modifier l'état des lots dans le ledger
- D'appliquer les règles métier (ex : un lot ne peut être transféré deux fois sans réception)

8.2 Méthodes du chaincode

Méthode	Effet sur le ledger	Appelée par
Créer un lot	Écrit un nouvel enregistrement	Agriculteur
Transférer un lot	Met à jour le propriétaire et ajoute un événement de transfert	Coopérative, transformateur, exportateur
Mettre à jour le poids	Modifie le poids en ajoutant une justification	Tout acteur (contrôle qualité)
Marquer comme exporté	Change le statut en "exporté"	Exportateur
Lire un lot	Retourne les données (lecture seule)	Tout le monde
Lire l'historique	Retourne tous les événements du lot (EUDR)	Tout le monde, notamment exportateur
Rechercher par agriculteur	Liste tous les lots d'un producteur	Agriculteur, ministère
Vérifier conformité EUDR	Calcule et retourne le statut	Exportateur, vérificateur public

8.3 Règles métier implémentées dans le chaincode

Règle	Description
Unicité du lot	Un identifiant ne peut être créé qu'une seule fois
Transférabilité	Seul le propriétaire actuel peut initier un transfert
Chaîne valide	On ne peut transférer que vers une organisation du consortium
Intégrité du poids	Le poids ne peut être modifié sans justification tracée
Horodatage forcé	Chaque transaction inclut automatiquement l'horodatage du bloc
Non-répudiation	Chaque transaction est signée par l'organisation expéditrice

9. API BACKEND

9.1 Rôle de l'API Gateway

L'API Gateway en Go assure :

- La réception des requêtes depuis les interfaces web (Next.js) et mobiles (React Native)
- L'authentification et l'autorisation via JWT (JSON Web Tokens)
- L'interaction avec Hyperledger Fabric via le fabric-sdk-go
- La gestion des utilisateurs dans PostgreSQL
- La génération des QR codes et rapports PDF
- La synchronisation des mobiles hors ligne

9.2 Endpoints API

Méthode	Endpoint	Description	Authentification
POST	/api/v1/auth/login	Connexion utilisateur	Non
POST	/api/v1/auth/register	Inscription coopérative/exportateur	Admin
POST	/api/v1/lot	Créer un nouveau lot	JWT (agriculteur)
GET	/api/v1/lot/{id}	Récupérer un lot par son ID	Optionnelle
GET	/api/v1/lot/{id}/history	Historique complet (EUDR)	Optionnelle
POST	/api/v1/transfer	Transférer un lot	JWT (coop/transfo/export)
PUT	/api/v1/lot/{id}/weight	Mettre à jour le poids	JWT
POST	/api/v1/lot/{id}/export	Marquer comme exporté	JWT (exportateur)
GET	/api/v1/verify/{id}	Vérification publique	Aucune
GET	/api/v1/eudr/{id}/report	Télécharger rapport EUDR PDF	JWT (exportateur)
GET	/api/v1/qrcode/{id}	Générer QR code	Optionnelle
POST	/api/v1/sync	Synchroniser lots hors ligne	JWT (agriculteur)
GET	/api/v1/dashboard/stats	Statistiques en temps réel	JWT (admin, ministère)

9.3 Format des requêtes/réponses

Création d'un lot (requête) :

- Identifiant agriculteur
- Nom agriculteur
- Coordonnées GPS (latitude, longitude)

- Poids (kg)
- Type de produit (cacao/café)
- Variété
- Date de récolte

Réponse succès :

- Statut ("created")
- Identifiant unique du lot
- QR code (image encodée en base64)
- Horodatage de la transaction

Vérification publique (réponse) :

- Identifiant du lot
- Origine complète (nom agriculteur, GPS, village)
- Liste chronologique des transferts
- Propriétaire actuel
- Statut de conformité EUDR
- Hash blockchain

9.4 Authentification et sécurité

Mécanisme	Description
JWT (JSON Web Token)	Généré après connexion, valide 24h, contient le rôle et l'organisation
Renouvellement automatique	Raffraîchi via token dédié
Chiffrement TLS	Toutes les communications en HTTPS
CORS	Restreint aux domaines autorisés (frontend Next.js, mobile)
Rate limiting	Limitation à 100 requêtes/minute par IP pour la vérification publique

10. FRONTEND WEB (NEXT.JS)

10.1 Architecture Next.js

Structure du projet :

```
chaincacao-web/
├── app/                          # App Router (Next.js 14+)
│   ├── (public)/                 # Routes publiques
```

```
│   │   ├── page.tsx              # Page d'accueil (/)
│   │   ├── verify/
│   │   │   └── [id]/
│   │   │       └── page.tsx      # Page de vérification (/verify/[id])
│   │   ├── about/
│   │   │   └── page.tsx          # À propos
│   │   └── contact/
│   │       └── page.tsx          # Contact
│   ├── (dashboard)/              # Routes authentifiées
│   │   ├── dashboard/
│   │   │   ├── page.tsx          # Tableau de bord (/dashboard)
│   │   │   ├── lots/
│   │   │   │   ├── page.tsx      # Gestion des lots
│   │   │   │   └── transfer/
│   │   │   │       └── page.tsx  # Transfert de lots
│   │   │   ├── reception/
│   │   │   │   └── page.tsx      # Réception de lots (scan QR)
│   │   │   └── eudr/
│   │   │       └── page.tsx      # Génération rapport EUDR
│   │   ├── login/
│   │   │   └── page.tsx          # Page de connexion
│   │   └── admin/                # Backoffice Ministère
│   │       ├── page.tsx          # Supervision
│   │       └── audit/
│   │           └── page.tsx      # Audit complet
│   ├── api/                      # API Routes (proxy vers Go)
│   │   └── [...path]/
│   │       └── route.ts          # Proxy dynamique
│   ├── layout.tsx                # Layout principal
│   └── globals.css               # Styles globaux
├── components/                   # Composants réutilisables
│   ├── ui/                       # Composants UI (shadcn/ui)
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   ├── dialog.tsx
│   │   └── ...
│   ├── qr-scanner.tsx            # Lecteur QR code
│   ├── lot-timeline.tsx          # Ligne du temps lot
│   ├── eudr-report.tsx           # Générateur rapport EUDR
│   └── map-view.tsx              # Cartographie interactive
├── lib/                          # Bibliothèques et utilitaires
│   ├── api-client.ts             # Client API (Go backend)
│   ├── auth.ts                   # Authentification NextAuth.js
│   └── utils.ts
├── hooks/                        # Hooks personnalisés
│   ├── use-auth.ts
│   └── use-lots.ts
├── types/                        # Types TypeScript
│   └── index.ts
├── public/                       # Assets statiques
```

```
│   ├── images/
│   └── favicon.ico
├── middleware.ts                # Middleware (routing protégé)
├── next.config.js               # Configuration Next.js
├── package.json
└── Dockerfile
```

10.2 Types de rendu Next.js utilisés

Page	Type de rendu	Justification
Page d'accueil (`/`)	**SSG (Static Site Generation)**	Contenu statique, généré au build, ultra-rapide
Vérification (`/verify/[id]`)	**ISR (Incremental Static Regeneration)**	Page statique régénérée après chaque mise à jour du lot (ex: toutes les 60s)
Tableau de bord	**SSR (Server-Side Rendering)**	Données en temps réel, authentification requise
Génération EUDR	**CSR (Client-Side Rendering)** + API	Formulaire interactif, téléchargement PDF
Backoffice	**SSR**	Données sensibles, contrôle d'accès

Configuration ISR pour `/verify/[id]` :

```
// Dans app/verify/[id]/page.tsx
export const revalidate = 60;  // Revalidation toutes les 60 secondes
export const dynamic = 'force-static';  // Génération statique à la
demande
```

10.3 Pages détaillées

10.3.1 Page d'accueil (`/`)

Type de rendu : SSG

Contenu :

- Présentation du projet ChainCacao
- Chiffres clés (40 000 familles, impact EUDR)
- Vidéo explicative
- Appel à l'action : scanner un QR code (lien vers `/verify`)
- Liens de connexion pour chaque rôle (coopérative, transformateur, exportateur, ministère)

10.3.2 Page de vérification publique (`/verify/[id]`)

Type de rendu : ISR (revalidation 60s)

Comportement :

Cas	Comportement
Premier accès	Génération statique de la page depuis les données Fabric
Mise à jour du lot (ex: transfert)	Page régénérée automatiquement dans les 60 secondes
ID inexistant	Page 404 personnalisée avec formulaire de recherche

Composants affichés :

1. **En-tête** : Identifiant du lot, statut (conforme/non conforme)
2. **Carte d'origine** : Coordonnées GPS, nom agriculteur, village, photo
3. **Ligne du temps interactive** : Chronologie des transferts
4. **Preuve blockchain** : Hash de transaction, lien explorateur
5. **Actions** : Télécharger PDF, partager, vérifier un autre lot

SEO : Next.js génère des métadonnées dynamiques pour chaque lot (title, description, Open Graph)

10.3.3 Tableau de bord Coopérative (`/dashboard`)

Type de rendu : SSR (authentification requise)

Composants principaux :

- **Liste des lots entrants** : Tableau avec filtres
- **Scanner QR code intégré** : Utilisation de la webcam
- **Formulaire de transfert** : Sélection d'un lot, choix du transformateur
- **Historique des transferts** : Traçabilité interne

10.3.4 Interface Exportateur (`/dashboard/eudr`)

Type de rendu : CSR (client-side rendering) + API

Fonctionnalités :

- Sélection multiple de lots (checkbox)
- Aperçu avant génération rapport
- Bouton "Générer rapport EUDR" → appel API Go
- Téléchargement automatique du PDF
- Envoi par email à l'acheteur européen

10.3.5 Backoffice Ministère (`/admin`)

Type de rendu : SSR + authentification élevée

Composants :

- **Cartographie** : Visualisation des lots par région (bibliothèque MapLibre ou Leaflet)
- **Graphiques** : Volume tracé par mois, conformité EUDR
- **Alertes** : Liste des lots suspects (pas de transfert depuis 30 jours)
- **Recherche avancée** : Auditeur de lots

10.4 API Routes Next.js (Proxy)

Les API Routes de Next.js servent de proxy vers l'API Gateway Go pour :

Avantage	Description
Simplification CORS	Même domaine, pas de problèmes cross-origin
Cache	Mise en cache des pages de vérification
Rate limiting	Protection au niveau Next.js avant appel Go
Middleware	Authentification centralisée

Exemple de fonctionnement :

```
Requête Next.js : fetch('/api/lot/TC-123')
       │
       ▼
API Route Next.js : /api/lot/TC-123
       │
       │ (proxy)
       ▼
API Gateway Go : http://go-backend:8080/api/v1/lot/TC-123
```

10.5 Authentification avec NextAuth.js

Configuration :

- Fournisseur : "Credentials" (email/mot de passe)
- Sessions : JWT
- Callbacks : Ajout du rôle et de l'organisation dans le token

Pages protégées : Toutes les routes sous `/dashboard` et `/admin` nécessitent authentification. Le middleware Next.js redirige automatiquement vers `/login` si non authentifié.

Rôles et redirections :

Rôle après connexion	Redirection vers
Coopérative	`/dashboard/reception`
Transformateur	`/dashboard/lots`
Exportateur	`/dashboard/eudr`
Ministère	`/admin`

11. APPLICATION MOBILE (REACT NATIVE)

11.1 Pages de l'application mobile

Onglet/Écran	Rôle	Utilisateurs
Accueil – Nouveau lot	Formulaire simplifié de création de lot	Agriculteur
Mes lots	Liste des lots créés avec statut de synchronisation	Agriculteur
Scanner	Lecture de QR code pour vérification rapide	Agriculteur, agent
Profil	Informations personnelles, synchronisation manuelle	Agriculteur

11.2 Fonctionnalités spécifiques au mobile

Capture GPS automatique :

- L'application récupère automatiquement les coordonnées GPS via le capteur du téléphone
- Si le GPS est désactivé, l'utilisateur est invité à l'activer
- En cas d'absence de signal, saisie manuelle possible (carte interactive)

Photo du lot :

- L'agriculteur peut prendre une photo de son lot à la récolte
- La photo est compressée et stockée localement, puis synchronisée
- La photo n'est pas stockée en blockchain (trop volumineuse) mais dans PostgreSQL avec lien vers le lot

Mode hors ligne complet :

- L'agriculteur peut créer des lots sans aucune connexion internet
- Les lots sont stockés dans SQLite avec horodatage local

- Une notification indique le nombre de lots en attente de synchronisation
- La synchronisation se fait automatiquement en arrière-plan dès que la connexion revient

Utilisation avec faible luminosité et poussière :

- Interface à fort contraste
- Boutons larges (adaptés aux doigts)
- Pas de gestes complexes (glisser, pincer)

11.3 Gestion de la synchronisation

Cas	Comportement
Connexion 4G/wifi présente	Envoi immédiat à l'API (via Next.js API routes ou directement Go)
Connexion interrompue	Stockage local, retransmission automatique à la reconnexion
Conflit (modification du même lot sur deux appareils)	Le système conserve le lot avec l'horodatage le plus récent, alerte (mode manuel)
Synchronisation massive (100+ lots)	File d'attente batch, upload progressif pour éviter l'engorgement réseau

12. EXIGENCES NON FONCTIONNELLES

Catégorie	Exigence	Niveau cible
Disponibilité	L'API et la blockchain doivent être accessibles	99,5 % (hors maintenance programmée)
Performance – écriture	Temps d'enregistrement d'un lot (API → blockchain)	Moins de 5 secondes
Performance – lecture	Temps de vérification d'un lot via QR code (Next.js ISR)	Moins de 2 secondes (cache)
Performance – recherche	Consultation de l'historique d'un lot	Moins de 2 secondes
Scalabilité – lots	Capacité du réseau Fabric	50 000 lots par an
Scalabilité – utilisateurs	Utilisateurs simultanés	500 connectés simultanés en phase pilote
Sécurité – transmission	Chiffrement	TLS 1.3 pour toutes les communications
Sécurité – stockage	Mots de passe	Hash (bcrypt)
Résilience – mobiles	Fonctionnement hors ligne	100 % des fonctionnalités de création disponibles

Catégorie	Exigence	Niveau cible
Conformité – données	RGPD (pour consommateurs européens)	Oui, avec droit à l'effacement (hors blockchain)
Conformité – export	EUDR	Preuve de traçabilité géographique obligatoire
SEO	Pages de vérification indexables	Oui (métadonnées dynamiques)

13. CONTRAINTES SPÉCIFIQUES

13.1 Contraintes terrain

Contrainte	Solution technique
Faible connexion internet en zone rurale	Mode hors ligne avec SQLite, synchronisation différée
Téléphones Android d'entrée de gamme	Application légère (taille < 30 Mo) sans animations lourdes
Alphabétisation variable des agriculteurs	Interface à base d'icônes, support vocal possible (optionnel)
Poussière et lumière variable	Interface à contraste élevé, boutons larges

13.2 Contraintes réglementaires

Contrainte	Périmètre
EUDR (Règlement européen anti-déforestation)	Applicable à tout cacao exporté vers l'UE à partir de 2025
Données personnelles (RGPD)	Pour les consommateurs européens traçant des lots
Loi togolaise sur la filière café-cacao	Conformité avec l'Office Togolais du Café et Cacao

13.3 Contraintes de déploiement Next.js

Élément	Solution
Hébergement Next.js	Vercel (recommandé pour ISR) ou auto-hébergé (Docker)
Cache ISR	Redis pour la revalidation (ou filesystem)
Build time	Moins de 5 minutes pour 10 000 pages statiques (ISR)
Fallback	Fallback blocking pour les IDs non encore générés

14. LIVRABLES PAR PHASE (HACKATHON)

Phase 1 – Présélection

Livrable	Description
Document de cadrage	Problème documenté avec données chiffrées, impact EUDR, solution proposée
Maquettes visuelles (Figma)	Interfaces pour agriculteur (mobile), coopérative (Next.js), exportateur, vérificateur public
Site web vitrine (Next.js)	Page d'accueil SSG, présentation du projet, problème, solution, ODDs
Architecture technique	Description de la blockchain Hyperledger Fabric + Next.js + Go
Plan de déploiement	Phases de déploiement au Togo (zone pilote → extension nationale)

Phase 2 – Demi-finale

Livrable	Description
Interface web Next.js fonctionnelle	Enregistrement d'un lot, transfert entre acteurs (coopérative → transformateur)
Page de vérification `/verify/[id]`	ISR, affichage de l'historique, SEO
Application mobile de base	Saisie terrain par l'agriculteur (création de lot, GPS, mode hors ligne)
Démonstration live	Flux complet : création lot sur mobile → synchronisation → transfert sur Next.js → vérification via `/verify/[id]`

Phase 3 – Finale

Livrable	Description
MVP complet	Next.js (App Router) + React Native + Go + Hyperledger Fabric
Flux complet démontrable	Devant le jury en temps réel avec plusieurs acteurs simulés
Documentation technique	Guide d'installation, architecture, API endpoints, déploiement Next.js
Guide utilisateur	Manuel pour chaque type d'acteur (français)
Pitch 10 minutes	Problème → Solution → Démo en direct (avec ISR visible) → Impact

15. CRITÈRES D'ACCEPTATION

Le projet est considéré comme livré et accepté si l'ensemble des critères suivants sont remplis :

#	Critère	Mode de vérification
1	Un lot peut être créé depuis l'application mobile avec capture GPS	Démonstration en zone rurale simulée
2	Le lot créé remonte automatiquement dans l'interface Next.js coopérative	Test d'intégration
3	La coopérative peut transférer le lot au transformateur via Next.js	Test manuel
4	Un QR code scanné redirige vers `/verify/[id]` et affiche l'historique complet	Test d'acceptation
5	La page `/verify/[id]` est statique et se recharge sans appel API (ISR)	Vérification Lighthouse/PageSpeed
6	Un rapport EUDR (PDF) peut être généré en un clic par l'exportateur	Test utilisateur
7	L'application mobile fonctionne entièrement hors ligne (création, stockage)	Test en mode avion
8	La synchronisation automatique se déclenche au retour du réseau	Test après coupure réseau
9	L'API Go répond en moins de 5 secondes pour une transaction d'écriture	Test de charge
10	La page de vérification est indexable (SEO) avec métadonnées dynamiques	Inspection Google Search Console
11	Le pitch mentionne explicitement l'impact sur 40 000 familles et la conformité EUDR	Évaluation jury

16. INDICATEURS DE PERFORMANCE (KPI)

Après déploiement en conditions réelles, les indicateurs suivants seront mesurés :

Indicateur	Cible à 6 mois	Méthode de mesure
Taux d'adoption par les coopératives dans zone pilote	> 80 %	Enquêtes terrain
Réduction des pertes liées aux fraudes sur pesées	-20 %	Comparaison avec données historiques
Temps moyen de chargement page `/verify/[id]` (First Paint)	< 500 ms	Lighthouse / Web Vitals

Indicateur	Cible à 6 mois	Méthode de mesure
Taux de pages `/verify/[id]` servies depuis le cache ISR	> 95 %	Logs Next.js
Pourcentage d'exportations certifiées conformes EUDR	> 95 %	Déclarations exportateurs
Coût moyen par transaction blockchain	< 0,001 USD (sur Fabric)	Monitoring Fabric
Temps d'enregistrement d'un lot (fin saisie → confirmation)	< 2 minutes (dont connexion)	Test utilisateur
Score SEO Google (page de vérification)	> 90/100	Lighthouse SEO

17. PLANNING DE DÉPLOIEMENT (POST-HACKATHON)

Phase pilote – Mois 1 à 3

Étape	Durée	Activités
Installation réseau Fabric	2 semaines	Déploiement des 4 organisations en environnement sécurisé
Déploiement Next.js	1 semaine	Vercel ou auto-hébergement Docker, configuration ISR
Formation pilote	1 semaine	Formation de 5 coopératives pilotes et 200 agriculteurs
Lancement zone pilote	6 semaines	Zone test : 5 villages (région des Plateaux)
Ajustements	2 semaines	Corrections et améliorations suite aux retours terrain

Extension – Mois 4 à 12

Étape	Durée	Activités
Extension vers 20 coopératives supplémentaires	4 mois	Déploiement progressif par zone géographique
Intégration transformateurs nationaux	2 mois	Raccordement des 5 principaux transformateurs
Optimisation ISR	1 mois	Configuration avancée du cache pour 10 000+ lots
Certification EUDR officielle	2 mois	Obtention de la reconnaissance par les autorités européennes

Étape	Durée	Activités
Généralisation	4 mois	Couverture nationale des 40 000 familles

Infrastructure cible

Composant	Solution
Next.js	Vercel Enterprise (ou auto-hébergé Kubernetes)
API Go	Cluster Kubernetes scaling automatique
Hyperledger Fabric	4 organisations sur serveurs dédiés (ou cloud)
PostgreSQL	Managed database (AWS RDS / Scaleway)
Stockage photos	Cloud Storage (AWS S3 / Cloudflare R2)

ANNEXES

Annexe A – Glossaire technique

Terme	Définition
Blockchain	Registre distribué immuable et sécurisé par cryptographie
Chaincode	Smart contract dans Hyperledger Fabric
Channel	Canal de communication privé entre certaines organisations Fabric
CSR	Client-Side Rendering (rendu côté navigateur)
EUDR	Règlement européen sur les produits sans déforestation (2025)
GPS	Système de positionnement par satellite
Hyperledger Fabric	Framework blockchain permissionné de la Linux Foundation
ISR	Incremental Static Regeneration (Next.js)
JWT	JSON Web Token – jeton d'authentification
Ledger	Registre immuable des transactions
Next.js	Framework React avec SSR/SSG/ISR
Ordre	Nœud responsable du consensus dans Fabric
Peer	Nœud qui détient une copie du ledger et exécute les chaincodes
SDK	Kit de développement logiciel
SSG	Static Site Generation (génération statique au build)
SSR	Server-Side Rendering (rendu côté serveur)
Transaction	Opération atomique qui modifie l'état du ledger

Annexe B – Responsabilités ODD couvertes

ODD	Contribution du projet
ODD 1 – Pas de pauvreté	Augmentation de la rémunération des agriculteurs (accès aux marchés premium)
ODD 2 – Faim zéro	Sécurisation des revenus agricoles, lutte contre la déforestation
ODD 8 – Travail décent	Réduction des fraudes, traçabilité transparente
ODD 12 – Consommation responsable	Traçabilité prouvée pour consommation éthique en Europe